UESTC 1005 - Introductory Programming
Lecture 8 Handout: Pointers (Continued)

Hasan T Abbas

Abstract

This document provides a detailed narrative and expanded context for the slides from Lecture 8. It covers the relationship between pointers and arrays, introduces dynamic memory allocation, and explores advanced uses of pointers with functions, including function pointers and pointers-to-pointers.

Contents

1 Recap: The Dangling Pointer

From our previous lecture, we know a pointer is a variable that stores the memory address of another variable. They are essential for efficient memory management, direct data access, and dynamic memory handling.

A "dangling pointer" is one of the most dangerous and common bugs in C. It is a pointer that points to a memory location that is no longer valid. This is different from a `NULL` pointer:

- **NULL Pointer:** A safe, intentional pointer that points to nothing (address 0). You can check for it.

```c
int *p = NULL; // Good practice: initialize
if (p != NULL) { /* ... */ }
```

- **Dangling Pointer:** A pointer that *used to* be valid, but the memory it points to has been deallocated (freed) or has gone out of scope (e.g., pointing to a local variable of a function that has returned).

Best Practice

Always initialise pointers, preferably to `NULL`. After you deallocate memory with `free()`, immediately set the pointer to `NULL`. This turns a dangerous dangling pointer into a safe, checkable `NULL` pointer.

```c
free(p);
p = NULL; // Now it's safe!
```

2 Pointers and Arrays

In C, pointers and arrays are very closely related. In most contexts, when you use the name of an array, it "decays" into a constant pointer to its very first element.

- `arr` is equivalent to `&arr[0]`.
- This pointer is **constant**. You cannot modify it: `arr++;` is a compile error.

2.1 Pointer Arithmetic

This is the key concept. When you add a number `i` to a pointer `p`, C does not add `i` bytes. It automatically scales the addition by the size of the data type the pointer points to.

`p + i` is equivalent to `(address in p) + (i * sizeof(*p))`

This is precisely why the array-access syntax `arr[i]` is **exactly equivalent** to the pointer-dereference syntax `*(arr + i)`.

2.2 Traversing and Modifying Arrays

You can use a pointer to iterate over and modify array elements.

```c
int arr[] = {78, 81, 88, 46, 28};
int *ptr = arr; // ptr points to arr[0]

for (int i = 0; i < 5; i++) {
    // *(ptr + i) is the same as ptr[i] or arr[i]
    printf("%d ", *(ptr + i));
}
// Output: 78 81 88 46 28
```

Listing 1: Traversing an array with a pointer

```c
int arr[] = {78, 81, 88, 46, 28};
int *ptr = arr;

// Modify arr[4] (value 28) to be 20
*(ptr + 4) = 20;

// The original array 'arr' is now changed.
// arr[4] is now 20.
```

Listing 2: Modifying an array via a pointer

3 Dynamic Memory Allocation

So far, our arrays (e.g., `int arr[10];`) have a fixed size that must be known when we compile the code. What if we don't know the size until the program is running?

3.1 Why Dynamic Memory? The Stack vs. The Heap

To solve this, we must understand C's two main memory areas:

- **The Stack:** Think of this as a "fast food counter." It's fast, automatic, and highly organized. When a function is called, its local variables (`int x;`, `int arr[10];`) are placed on the Stack. When the function returns, this memory is *automatically* cleaned up. It's fast, but it's small and the size of everything must be known at compile time.
- **The Heap:** Think of this as a large "restaurant dining hall." It's a big, open pool of memory. You can't just take memory; you must *manually* ask for it (`malloc`). You can ask for any size at any time. When you're done, you *must* manually return it (`free`). It's flexible, but it's slower and managed entirely by you, the programmer.

Table 1: Stack vs. Heap Summary

Feature	**The Stack (Automatic)**	**The Heap (Manual)**
Allocation	Automatic (on function call)	Manual (you must call `malloc()`)
Deallocation	Automatic (on function return)	Manual (you must call `free()`)
Size	Small, Fixed	Large, Flexible
Speed	Very Fast	Slower
Managed By	Compiler	**You (The Programmer)**
Typical Bug	Stack Overflow	Memory Leak / Dangling Pointer

3.2 The Tools: `malloc()` and `free()`

To use the Heap, we use two functions from `<stdlib.h>`:

- `void* malloc(size_t size)`: "Memory Allocate." Asks the Heap for `size` bytes. `size_t` is a special unsigned integer type for memory sizes (it's what `sizeof()` returns). It returns a generic `void*` pointer to the new memory, or `NULL` if it fails.
- `void free(void* ptr)`: Returns a block of memory (that you got from `malloc`) back to the Heap.

3.3 The Dynamic Array: Full Pattern

Here is the complete pattern for creating and using a dynamic array.

```c
#include <stdio.h>
#include <stdlib.h> // For malloc() and free()

int main() {
    int n = 10; // This 'n' could come from user input!
    int *arr;

    // Request memory for 'n' integers
    arr = (int*)malloc(n * sizeof(int));

    // Always check if malloc failed!
    if (arr == NULL) {
        printf("Error: Memory allocation failed.\n");
        return 1; // Exit
    }

    // 'arr' now acts just like a normal array
    for (int i = 0; i < n; i++) {
        arr[i] = i * 2;
    }

    // Return the memory to the Heap when done
    free(arr);

    // BEST PRACTICE: Prevent a dangling pointer
    arr = NULL;

    return 0;
}
```

Listing 3: The 4-step pattern for dynamic memory

This is the direct connection: calling `free(arr)` deallocates the memory, but `arr` itself still holds the old address. At that moment, it becomes a **dangling pointer**. Setting it to `NULL` fixes this.

4 Pointers and Functions

Pointers become incredibly powerful when used with functions.

4.1 Simulating Pass-by-Reference

C is a "pass-by-value" language. A function always gets a *copy* of its arguments. You cannot change a caller's variable. However, you can pass a *copy of a pointer*. This pointer still points to the *original* data, allowing the function to "reach back" and modify it.

```c
void increment(int *val) {
    // 'val' is a copy of the address of 'x'
    // '*val' accesses the *original* 'x'
    (*val)++;
    // Parentheses are crucial! *val++ would increment
    // the pointer, not the value it points to.
}

```

```c
int main() {
    int x = 5;
    increment(&x); // Pass the address of x
    printf("%d\n", x); // Outputs: 6
    return 0;
}
```

Listing 4: Using a pointer to modify a caller's variable

4.2 Pointers to Functions

Just as a pointer can store the address of a variable, a **function pointer** can store the address of a function. This allows you to treat functions as data: you can pass them as arguments, store them, and call them dynamically.

Syntax: `return_type (*pointer_name)(parameter_list);`

```c
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }

// This function takes a function pointer as an argument
void executeOperation(int (*operation)(int, int), int x, int y) {
    printf("Result: %d\n", operation(x, y));
}

int main() {
    // Pass the 'add' function
    executeOperation(add, 10, 5);       // Outputs: Result: 15

    // Pass the 'subtract' function
    executeOperation(subtract, 10, 5);  // Outputs: Result: 5
    return 0;
}
```

Listing 5: Example of a function pointer

Function pointers are essential for:

- **Callbacks:** Giving a function another function to "call back" when an event occurs (e.g., a button click).
- **Dynamic Function Selection:** Choosing at runtime which function to execute (like the calculator example).
- **Custom Comparators:** Telling a generic sorting function (like `qsort`) *how* to compare your specific data.

4.3 `const` Pointers in Functions

Using `const` in function arguments is good practice to prevent accidental modification.

- `const int *p`: A pointer to a `const int`. You cannot change the value `*p`, but you can change the pointer `p` itself. Use this to pass an array for *reading only*.
- `int * const p`: A `const` pointer to an `int`. You can change `*p`, but you cannot change where `p` points.

5 Pointers to Pointers (Double Pointers)

A "pointer to a pointer" is a variable that stores the memory address of *another pointer*.

```c
int x = 100;
int *p = &x;      // 'p' points to 'x' (type: int*)
int **pp = &p;    // 'pp' points to 'p' (type: int**)
```

To access the value `x`, you must dereference twice:

- `pp` holds the address of `p`.
- `*pp` dereferences `pp` and gives you `p`.
- `**pp` dereferences `p` (which is `*pp`) and gives you `x`.

`printf("%d\n", **pp);` *Outputs 100*

5.1 Why Use Double Pointers?

This concept seems complex, but it solves a crucial problem. We know we can pass a pointer to a function to modify the *original value* (like `increment`).

What if we want a function to modify the *original pointer itself*? For example, what if we want a function to allocate memory and give that new memory back to the caller?

You must pass a pointer *to that pointer*.

```c
#include <stdlib.h>

// This function takes a *pointer to a pointer*
void allocateMemory(int **ptr_to_ptr, int size) {
    // *ptr_to_ptr is the 'p' from main
    *ptr_to_ptr = (int*)malloc(size * sizeof(int));

    if (*ptr_to_ptr != NULL) {
        // Set a value in the new memory
        (*ptr_to_ptr)[0] = 25;
    }
}

int main() {
    int *p = NULL; // p starts as NULL

    // We pass the *address of p* (&p)
    allocateMemory(&p, 10);

    // 'p' itself has now been modified by the function.
    // It is no longer NULL and points to new memory.
    if (p != NULL) {
        printf("%d\n", p[0]); // Outputs: 25

        free(p); // Always free allocated memory
        p = NULL;
    }
    return 0;
}
```

Listing 6: Using a double pointer to modify a caller's pointer

This is a very common and powerful pattern in C. Another major use is for creating dynamic 2D arrays (an array of pointers, where each pointer points to a row).